Компьютерные науки и анализ данных

Yandex

Faculty of Computer science
Higher School of Economics

Программирование на Python

лектор: Сергей Горшков

План

- Хэш. Хэш-таблица. Требование неизменяемости объекта хэширования. Понятие коллизии
- Абстрактные типы данных: множество, словарь (основы)
- Встроенные типы данных dict, set, frozenset
- Модуль collections: Counter, defaultdict, OrderedDict, ChainMap (последний на самостоятельное изучение)
- Формат json

Хэш-таблица

- Хэш-таблица – массив большого размера K. Каждому *неизменяемому* объекту можно сопоставить число M от 0 до K-1 вкл и поместить этот объект в ячейку списка с индексом M. Пример – взятие остатка по модулю K для целых чисел
- Произвольные типы данных – можно представить последовательность байт как число и считать хэш-функцию для полученного числа
- Разные значения могут иметь одинаковый хэш. Такая ситуация называется коллизией. Методы разрешения будут рассмотрены в курсе алгоритмов
- h(x) - hash: if x1 == x2 => h(x1) == h(x2)

Работа с хэшами в Python

- Каждый неизменяемый объект в Python имеет заранее насчитанный хэш, который подсчитывается один раз при его создании.
- Хэш от небольшого натурального числа – само число
- Изменяемые типы хэшей не имеют: для список, например, изменение одного элемента привело бы к полному перехэшированию, что катастрофически замедлило бы работу. Поэтому у изменяемых объектов нет хэша и они не могут быть добавлены в множество. Если всё же хотим положить *массив* в множество, то необходимо будет создать из списка кортеж и использовать его.

Множество

- Множество – набор объектов без определенного порядка
- В множество можно добавлять и удалять элементы, проверять принадлежность элемента множеству и перебирать все объекты множества
- Первые три типа операций должны выполняться за O(1)
- Можно совершать операции из теории множеств: пересечение, объединение …
- Само множество является изменяемым объектом. Есть отдельный тип неизменяемого множества в Python (его уже можно будет сделать элементом множества)
- **Элементы в множестве уникальные (встроенного типа мультимножества в Python нет)**

set

- Создание: `set()`, или перечисление `{1, 2, 3}` (пустое множество – **только** `set()`)
- Создание множества из итерируемых объектов: `set(range(2, 22, 3))`
- Множество может содержать объекты разных типов
- Множество является итерируемым объектом (порядок элементов не задан)
- Размер множества `len`, проверка наличия элемента в множестве (возвращает True/False) – `elem in my_set` (отсутствия – `elem not in my_set`)
- Можно применять map, filter, …
- Добавление элемента в множество – `add`: `my_set.add(elem)`
- Удаление:
  - remove – если не было такого элемента, то будет ошибка
  - discard – – || –, то состояние множества не изменится

Higher School of Economics
Faculty of Computer science

Операции над set. frozenset

- & (пересечение), | (объединение), - (разность), ^ (симметрическая разность)
- Модифицирующие inplace аналоги &=, |=, -=, ^=
- Определены все 6 операций сравнения (`A<B` – "A есть подмножество B" и т.п.)
- Все эти операции выполняются за время, пропорциональное числу элементов в множествах
- Есть методы, полностью аналогичные этим операторам.
- `{i**2 for i in range(7)}` – есть генератор множеств

- frozenset – неизменяемый set. Ровно такая же разница как между списками и кортежами

Словари (дикты, мапы)

- Структура данных, хранящая ключ-значение
- Объекты-ключи уникальны и каждому ключу сопоставлено ровно 1 значение
- `d = {}, d = dict()` – создание пустого словаря
- `d = {"key1": 1, 2: "value2", "empty": None}`
- `d = dict(short='dict', long='dictionary')`
- `d = dict([(1, 1), (2, 4)])`
- `d = dict.fromkeys(['a', 'b'], 100)`
- `d = {a: a ** 2 for a in range(7)}`– генераторы словарей
- Принадлежность ключа словарю – `in`, как и для множеств
- Словарь является iterable, возвращает неупорядоченные ключи

Работа со словарями в Python

- Удаление элемента по ключу: `del d['key']`. Это не функция, скобки не ставятся!
- `d.pop(key[, default])` - удаляет ключ и возвращает значение. Если ключа нет, возвращает default (по умолчанию бросает исключение).
- `d.items()` – возвращает iterable из кортежей ключ-значение
- `d.keys()` - возвращает ключи в словаре, `d.values()` – значения
- Получение элемента по ключу: `d['key']` – если нет, то исключение
- `d.get('key' [,default=None])` – –||–, то возвращается default
- `d.setdefault(key[, default])` – –||–, то создает ключ со значением default
- `dict.update([other])` - обновляет словарь, добавляя пары key-value из other. Существующие ключи перезаписываются. Возвращает None. `|` , `|=` с Python>=3.9
- Распаковка словаря: **kwargs

collections.OrderedDict

- Упорядоченность элементов по добавлению в словарь с Python3.7 (статья) (место в документации) (но для множества это не работает 🤷)
- od.popitem(last=True) – возвращает и удаляет последнюю / первую (last=False) пару ключ-значение
- od.move_to_end(key, last=True) – перемещает ключ в конец/начало
- Поддерживает reversed(), сравнение на равенство учитывает порядок добавления, в отличие от dict
- На этом может быть построен LRU кэш
- @functools.lru_cache(maxsize=128, typed=False) - декоратор, который сохраняет результаты maxsize последних вызовов

collections.defaultdict

- defaultdict позволяет задавать значение по умолчанию у объекта
- `s= [('yellow', 1),('blue', 2),('yellow', 3),('blue', 4),('red', 1)]`

```
d = defaultdict(list)
for k, v in s:
    d[k].append(v)
sorted(d.items())
```

```
d = {}
for k, v in s:
    d.setdefault(k, []).append(v)
sorted(d.items())
```

```
[('blue', [2, 4]), ('red', [1]), ('yellow', [1, 3])]
```

-
```
s = 'mississippi'
d = defaultdict(int)
for k in s:
    d[k] += 1
sorted(d.items())
```

collections.Counter

- Позволяет считать число объектов из iterable
- `c = Counter(a=4, b=2, c=0, d=-2)`
  `sorted(c.elements())` – можно просто получить все элементы
- `c.most_common([n])` – выводит топ-n самых часто встречаемых элементов
-
```
c = Counter(a=4, b=2, c=0, d=-2)
d = Counter(a=1, b=2, c=3, d=4)
c.subtract(d)
c
Counter({'a': 3, 'b': 0, 'c': -3, 'd': -6})
```
- `c.update(another)` – обновление аналогично

json

- JSON (JavaScript Object Notation) представляет собой одну из двух структур:
  - Набор пар ключ: значение (в Python – словарь) – `{}`
  - Упорядоченный набор значений (в Python – список) – `[]`
- Элементы могут быть эти, а также:
  - Число (целое или вещественное)
  - Литералы – true, false, null
  - Строка: набор символов, могут быть escape-последовательности и Unicode
- Модуль json:
  - json.dump – сериализует в поток, dumps – в строку
  - Десериализация – json.load и json.loads соответственно

Пример json

```
{
    "firstName": "Иван",
    "lastName": "Иванов",
    "address": {
        "streetAddress": "Московское ш.,
101, кв.101",
        "city": "Ленинград",
        "postalCode": 101101
    },
    "phoneNumbers": [
        "812 123-1234",
        "916 123-4567"
    ]
}
```

Красивое форматирование:

`json.dumps(indent=4, ensure_ascii=False).`

Второе условие – чтобы не-ASCII символы были записаны человекочитаемо

Консольная утилита jq

Faculty of Computer Science
Higher School of Economics

sgorshkov@hse.ru
tg: @Seragor98